# LAlgo Lexical Spec: Fixes and Summary

Fixes for the LAlgo Language Design Specification (Group 4, COSC 303). Each fix states the problem, why it matters, and drop-in text you can paste into the spec.

## Summary of fixes

| # | Problem | What was done |
|---|---|---|
| 1 | Lowercase keywords (`while`) become plain IDENT tokens | Keywords are uppercase-only, with a `suspectedKeyword` flag |
| 2 | `2low` is invalid in 2.2 but two valid tokens in 2.3.11 | Made it a lexical error |
| 3 | Newlines and statement boundaries are unclear | Newlines are still discarded, but tokens carry a `precededByNewline` flag |
| 4 | Σliteral vs. comments, and space belongs to two classes | Clarified Σ and allowed printable ASCII in comments |
| 5 | Gaps for `3.`, `.5`, leading zeros, and range | Wrote down the backtracking and the rules |
| 6 | Risky NOT precedence | Moved NOT below relational operators and fixed the table |
| 7 | No token table, source position, or error recovery | Added a new section 2.3.12 |
| 8 | δ, master DFA, quotes in literals, 2.9, and `^` | One or two lines of clarification each |

**Two decisions for the group to discuss:** the case rule in Fix 1 and the newline flag in Fix 3. Opinions can differ here, and both have tradeoffs.

## Fix 1: Lowercase keywords

**Problem:** keywords are uppercase, but `while` and `if` are valid identifiers under the identifier regex. So `while (x < 5) {}` comes out as IDENT, IDENT, and the parser is the one that blows up.

**Decision:** keywords are uppercase-only, but the lexer helps the student when they type lowercase.

**Why this choice:** making keywords case-insensitive would also reserve `input`, `output`, `string`, and `array`, which students often use as variable names. A warning flag has a smaller side effect and leaves the DFA untouched.

**Add to 2.5:**

> Keywords are recognized only in their uppercase spelling. An identifier whose lowercase form equals a keyword (for example `while`, `If`) is emitted as IDENT with a `suspectedKeyword` flag, so that later phases can report "Did you mean WHILE?". Such an identifier is not a lexical error and is not reserved.

## Fix 2: `2low` and `5thSample`

**Problem:** in 2.2, `2low` is an invalid identifier. But in 2.3.11, the lexer emits INTEGER `2` and IDENT `low`, which are both valid tokens. The two sections contradict each other.

**Why this fix:** making it a lexical error makes both sections agree and gives the student a clearer error message.

**Replace the bullet in 2.3.11 (Number-identifier boundaries):**

> **Number-identifier boundaries.** A numeric literal immediately followed by an alphabetic character or underscore (for example `5thSample`, `2low`) is a lexical error: `Malformed number '2low'`. The scanner reports the whole run of `[A-Za-z0-9_]` as one bad lexeme and resumes after it.

**In 2.2, replace the comment with:** `2low // Lexical error: number followed by letter`

## Fix 3: Newlines and statement boundaries

**Problem:** the lexer discards newlines and no semicolon is required, so it is unclear where a statement ends.

**Decision:** no NEWLINE token. Instead, each token carries a flag.

**Why:** a NEWLINE token would break your own style of putting `}` and `ELSE IF` on separate lines. The flag keeps the syntax free-field while still giving the parser the information if it needs it.

**Add to 2.10:**

> Line feeds are discarded and never emitted as tokens. Each token instead records `precededByNewline` (TRUE if at least one line feed separates it from the previous token). The future parser may use this flag to resolve statement boundaries without making line breaks significant.

## Fix 4: Σliteral and comments

**Problem:** Σ is described as "partitioned", but space (ASCII 32) is in both Σwhitespace and Σliteral. It is also unclear whether `&` or `#` is legal inside a comment, since they are said to be allowed only in quoted literals.

**Why this fix:** it removes a contradiction a reader would easily notice.

**Replace Σwhitespace and Σliteral in 2.1:**

> **Whitespace (Σwhitespace)**: space (ASCII 32), horizontal tab (ASCII 9), line feed (ASCII 10).
>
> **Literal/comment-only symbols (Σliteral)**: printable ASCII 33 to 126 not already in another class and other than backslash. These appear only inside quoted literals and comments.
>
> Space and horizontal tab are also permitted inside string literals, character literals, and comments, even though they belong to Σwhitespace.

Replace "partitioned" with "organized into lexical classes".

**Add to 2.7:**

> Comment bodies may contain any printable ASCII character (32 to 126) and horizontal tab. Block comments may also contain line feeds. Any other character inside a comment is a lexical error.

## Fix 5: Numeric literals

**Problem:** nothing is said about `3.`, `.5`, `007`, or very large numbers. For `3.`, the DFA stops in q2, which is non-accepting, so it looks like a bug unless the backtracking is written down.

**Add to 2.3.3:**

> - **Trailing dot.** `3.` is not a DECIMAL. The scanner backtracks and emits INTEGER `3` followed by DOT.
> - **Leading dot.** `.5` is DOT followed by INTEGER `5`; it is not a DECIMAL. A decimal needs at least one digit on each side of the point.
> - **Leading zeros.** Allowed and ignored in value (`007` = 7, `00.5` = 0.5).
> - **Range.** INTEGER literals must fit in 32-bit signed range (maximum 2147483647). A larger literal is a lexical error. Because signs are separate operators (2.3.11), the magnitude 2147483648 is also rejected. DECIMAL literals use IEEE 754 double precision.

## Fix 6: NOT precedence

**Problem:** in the old table, NOT binds tighter than relational operators. So `NOT found == TRUE` becomes `(NOT found) == TRUE`, which a beginner does not expect.

**Why this fix:** NOT moves below relational operators, like in Python, so `NOT found == TRUE` means `NOT (found == TRUE)`, as a reader would expect.

**Replace the table in 2.11.1 and place it right after the heading:**

| Rank | Operator | Description | Associativity |
|---|---|---|---|
| 1 | `( )` `[ ]` | Grouping, indexing | Left-to-right |
| 2 | `^` | Exponentiation | Right-to-left |
| 3 | unary `-` | Sign inversion | Right-to-left |
| 4 | `*` `/` `%` `DIV` | Multiplicative | Left-to-right |
| 5 | `+` `-` | Additive | Left-to-right |
| 6 | `<` `<=` `>` `>=` | Relational inequalities | Left-to-right |

| Rank | Operator | Description | Associativity |
|---|---|---|---|
| 7 | `==` `!=` | Relational equalities | Left-to-right |
| 8 | `NOT` | Logical negation | Right-to-left |
| 9 | `AND` | Conjunction | Left-to-right |
| 10 | `OR` | Disjunction | Left-to-right |

In LaTeX, use `\begin{table}[H]` (package `float`) so the table does not float to another page. In 2.4.2, replace "Highest" with "Highest among logical operators".

## Fix 7: Token table, position, and error recovery

**Problem:** the trace shows `TOKEN(IDENT, "count")`, but there is no complete list of token types, no line and column, and nothing about what the lexer does after an error.

**Why it matters:** tracing is LAlgo's selling point. To trace execution back to a source line, every token must carry its line and column.

**New 2.3.12 Token Structure and Error Handling:**

> Every token is a record `(type, lexeme, line, column, precededByNewline)`. Lines and columns start at 1, and a tab counts as one column.
>
> | Class | Token types |
> |---|---|
> | Identifier | IDENT |
> | Literals | INTEGER, DECIMAL, STRING, CHAR, TRUE, FALSE |
> | Type keywords | KW_INTEGER, KW_DECIMAL, KW_CHARACTER, KW_BOOLEAN, KW_ARRAY, KW_STRING |
> | Control/IO keywords | KW_IF, KW_ELSE, KW_WHILE, KW_FOR, KW_INPUT, KW_OUTPUT |
> | Operator keywords | DIV, AND, OR, NOT |
> | Arithmetic | PLUS, MINUS, STAR, SLASH, PERCENT, CARET |
> | Relational/assign | ASSIGN, EQ, NEQ, LT, GT, LE, GE |
> | Delimiters | LPAREN, RPAREN, LBRACKET, RBRACKET, LBRACE, RBRACE, COMMA, SEMICOLON, COLON, DOT |
> | Noise | NOISE_SET, NOISE_THE |
> | Future-reserved | RES_ALGORITHM, RES_RETURN, RES_TRACE, RES_INVARIANT |
> | Special | EOF |
>
> Whitespace and comments produce no token. On a lexical error (unterminated string, character, or block comment; malformed number; out-of-range integer; illegal character; lone `!`), the scanner records the error with its line and column, skips the offending lexeme, and continues. All errors are reported together, and EOF is always the last token.

## Fix 8: Small clean-ups

**δ total or partial.** Add to 2.3: "Diagrams show only defined transitions; every missing transition goes to an implicit non-accepting error state, so δ is total." *Why:* the definition is total but the drawings are partial.

**Master DFA.** Rename "Master Lexer Dispatch Automaton" to "Master Lexer Dispatch Procedure" and add "composed of the DFAs below; not itself a single DFA." Remove "whitespace loops at q0" from 2.3.1 and refer to 2.3.8 instead. *Why:* the dispatch returns to q0 after emitting, so it is not really one DFA.

**Quotes inside literals.** Add to 2.3.11: "Because escape sequences are not defined, a string cannot contain `\"` and a character literal cannot be `'`. Escapes are future work." *Why:* it is safer to admit the limitation than to add escapes whose DFA has not been designed yet.

**2.9 "algorithm parameters/bodies".** Replace with "grouping and call-like constructs (future)". *Why:* it contradicts the claim that ALGORITHM has no semantics yet.

`^` **rationale.** Add to 2.4.1: "`^` denotes exponentiation, following mathematical and textbook pseudocode notation; unlike C/Java, it is not bitwise XOR." *Why:* LAlgo is about moving from pseudocode to real code, so the difference should be explained.

## Next steps

1. Discuss Fix 1 and Fix 3 with the group.
2. Apply Fixes 1 to 4 first, since they are contradictions inside the document itself.
3. Add Fix 7, and include a worked binary-search example with its full token stream so any bug in the spec shows up right away.